Sorted Squared Array

AlgoExpert · Easy · Arrays

The problem

- Given a sorted array (may contain negatives) return the squares in sorted order.
- Two pointers from both ends fill the result from the back.

Intuition

- The largest squares sit at the two ends of a sorted array, because big negatives square to big positives.

Approach

- 1. Put pointers at both ends
- 2. Compare absolute values
- 3. Write the larger square into the result from the back
- 4. Move that pointer inward

Java solution

```java
public class SortedSquaredArray {
    public static int[] sortedSquaredArray(int[] array) {
        int[] out = new int[array.length];
        int l = 0, r = array.length - 1;
        for (int k = array.length - 1; k >= 0; k--) {
            if (Math.abs(array[l]) > Math.abs(array[r])) { out[k] = array[l] * array[l]; l++; }
            else { out[k] = array[r] * array[r]; r--; }
        }
        return out;
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Recap

- The largest squares sit at the two ends of a sorted array, because big negatives square to big positives.
- Complexity: O(n)